



DEPARTAMENTO
DE INDUSTRIAS

UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

LABORATORIO 3

AUTOMATIZACIÓN DE PROCESOS CON N8N

Lorenzo Ruiz

Ingeniería Civil Industrial, 202304619-1, lflandez@usm.cl

Repositorio: <https://github.com/LorenzoRF1/ICN292-Lab3-Ruiz-Lorenzo.git>

Parámetro	Valor
S	026
Umbral de monto U	\$56.000
Plazo máximo D	21 días

SEPTIEMBRE, 2026

CAMPUS VITACURA

Resumen ejecutivo

Se construyó un workflow en n8n que busca automatizar el triage de las solicitudes del canal de venta en línea de AndesHogar SpA. La solución tiene tres flujos: triage (con 12 nodos) que se encarga de recibir solicitudes, consultar valor UF, decidir ruta, guardar el resultado y responder; otro emisor, que se encarga de enviar las consultas; y uno de resumen, que junta todos los datos del día y manda un correo.

Con $S = 026$ resultó en $U = \$56,000$ y $D = 21$ días, las quince solicitudes quedaron en cuatro aprobaciones, siete revisiones en bodega y cuatro rechazos, sin datos inválidos, con un total de \$1.251.820 y una tasa de aprobación automática de 26,7%. Cada solicitud se procesa en unos dos segundos, frente a los días hábiles que demora actualmente. De cada una queda guardado su identificador, SKU, monto, ruta, motivo y los valores de U y D usados, por lo que se puede revisar fácilmente en caso de algún error.

Las pruebas de la Parte C muestran hasta donde es capaz el sistema. Las reglas se aplican en el orden correcto y los casos límite funcionan: una solicitud con monto igual a U , o con días iguales a D , queda aprobada. Por otro lado montos escritos en palabras generan errores y una solicitud con un SKU inexistente se aprueba directamente si cumple las reglas.

Se recomienda seguir usando el workflow con una revisión periódica, configurar un Error workflow cuando el flujo falle, y mientras tanto corregir y cerrar los huecos que posea el sistema actual.

Todo lo anterior se puede revisar con los tres archivos json y el registro CSV, junto con las capturas en el repositorio GitHub:<https://github.com/LorenzoRF1/ICN292-Lab3-Ruiz-Lorenzo>.



TABLA DE CONTENIDOS

1	Parte A Flujo de triage	3
1.1	Flujo	3
1.2	Switch	3
1.3	Paginas Externas	3
1.4	Tabla de Resultados	4
1.5	URL de prueba y de produccion	4
2	Parte B Flujo programado del resumen	4
2.1	Estructura del resumen	4
3	Parte C	5
3.1	Entradas Invalidas	5
3.2	Los tres valores de On Error	6
3.3	Error Workflow	6
3.4	Casos Bordes creados	6
3.5	Error de tipo, a proposito	7
3.6	Los dos casos que el flujo no pudo resolver	7
3.7	Recomendacion sobre el MVP	7
	Bibliografia	8

1 Parte A Flujo de triage

1.1 Flujo

El workflow de triage posee 12 nodos en total, la clasificacion de cada elemento se realizo unicamente con un nodo Switch en modo Rules junto con cuatro nodos Edit field siguiendo cada ruta; Rechazo, Revision, Datos invalidos, y se opto por usar la opcion FallBack Output para la ruta de aprobacion.



Figura 1: Arquitectura del flujo de triage

El orden se escogió debido a que el nodo HTTP Request (Encargado de extraer el valor de la UF) es necesario antes de el nodo Preparar, ya que este construye el cuerpo del sistema, para posteriormente el Switch poder separar caso por caso.

Otro dato a señalar es que se opto por registrar todos los datos antes de enviar el correo, ya que si el correo falla, la decisión ya quedo guardada en el registro.

1.2 Switch

En este nodo se fue poniendo uno por uno y en orden las distintas reglas estipuladas por la politica de la empresa, la primera que se cumple fija la ruta y detiene la evaluacion. Se opto por usar la salida de respaldo para todos los datos que no cumplen ningunas de las condiciones, correspondiendo a la ruta de APROBACIÓN.

#	Condición	Compara con	Salida
1	body.id_solicitud · String is empty	—	CAMPO_INVALIDO
2	body.monto.toNumber() · ≤	0	MONTO_INVALIDO
3	body.dias_desde_compra.toNumber() · >	21 (= D)	RECHAZO_PLAZO
4	body.estado_producto · is equal to	danado_por_uso	ESTADO_RECHAZO
5	body.monto.toNumber() · >	56000 (= U)	REVISION_MONTO
6	body.estado_producto · is equal to	con_fallas	REVISION_FALLA
—	Fallback Output	—	APROBACION

1.3 Paginas Externas

Se consulta en <https://mindicador.cl/api> mediante HTTP Request. Del JSON de respuesta se usa el campo uf.valor, que se almacena como uf_valor y permite expresar cada monto en UF mediante $\text{monto_uf} = \text{monto} / \text{uf_valor}$. En la ejecución el valor fue \$40.991,75 por UF.

1.4 Tabla de Resultados

Cuadro 1: Resultado del triage de las 15 solicitudes, sacados de la Data Table.

ID	Monto	Días	Estado	Ruta	Motivo
DEV-1115	19.990	3	sin_abrir	APROBACION	En plazo y bajo el umbral
DEV-1116	69.980	5	abierto_sin_uso	REVISION	Monto 69.980 > U
DEV-1117	24.990	12	con_fallas	REVISION	Producto con fallas
DEV-1118	79.990	4	sin_abrir	REVISION	Monto 79.990 > U
DEV-1119	45.990	25	danado_por_uso	RECHAZO	Fuera de plazo: 25 días
DEV-1120	89.990	2	con_fallas	REVISION	Monto 89.990 > U
DEV-1121	289.990	9	sin_abrir	REVISION	Monto 289.990 > U
DEV-1122	45.990	2	sin_abrir	APROBACION	En plazo y bajo el umbral
DEV-1123	39.980	15	danado_por_uso	RECHAZO	Producto dañado por uso
DEV-1124	49.980	1	abierto_sin_uso	APROBACION	En plazo y bajo el umbral
DEV-1125	34.990	22	sin_abrir	RECHAZO	Fuera de plazo: 22 días
DEV-1126	79.990	8	con_fallas	REVISION	Monto 79.990 > U
DEV-C01	9.990	1	sin_abrir	APROBACION	En plazo y bajo el umbral
DEV-C02	289.990	2	sin_abrir	REVISION	Monto 289.990 > U
DEV-C03	79.990	29	abierto_sin_uso	RECHAZO	Fuera de plazo: 29 días

Los resultados se pueden resumir en 4 aprobaciones, 7 revisiones, 4 rechazos y 0 datos inválidos. El monto total procesado fue \$1.251.820 y la tasa de aprobación automática, 26,7 %.

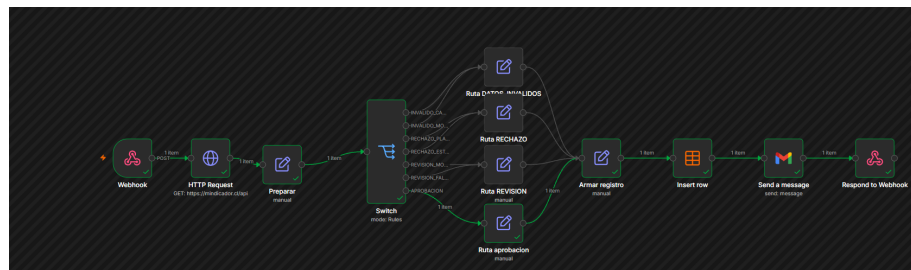


Figura 2: Workflow de ruta aprobacion

1.5 URL de prueba y de produccion

El sistema n8n publica dos URL para el mismo webhook. La Test URL que solo acepta una llamada despues de ejecutar siendo de mucha ayuda a la hora de depurar y construir el sistema. La Production URL se necesita tener el workflow publicado y atiende a todas las solicitudes que le entregan, registrando cada una en Executions.

La corrida de 15 solicitudes y las pruebas de la parte C se realizaron con la Production URL exceptuando la prueba de DEV-C01 que se realizo con la Test URL.

2 Parte B Flujo programado del resumen

2.1 Estructura del resumen

El workflow ICN292-Lab3-Ruiz-Lorenzo-resumen tiene ocho nodos en total, teniendo un lector del registro, filtrado de fecha, un IF para determinar si hubo o no solicitudes, las ramas de

resumen, y aviso. El Summarize agrupa cada ruta y calcula el conteo de `id_solicitud` para saber contabilizar cuantas solicitudes fueron en cada ruta, ademas de la suma de monto. El nodo Aggregate junta las tres filas en un único ítem, haciendo de esta forma que se envíe solamente un mensaje a excepción de uno por ruta.

Ruta	Solicitudes	Monto
APROBACION	4	\$125.950
REVISION	7	\$924.920
RECHAZO	4	\$200.950
Total	15	\$1.251.820

Sobre la frecuencia, el flujo se ejecuta una vez al día, a las 22:00. Se escogió esta hora ya que el servicio al cliente ya dejo de operar y el resumen llega con la información de todo el día. Si optáramos por usar una frecuencia horaria haria que gran parte de los correos estén vacíos o sean tantos que sean molestos de leer.

Ademas como hicimos uso de un nodo filtro, activando la opción de Always Output Data. Ya que gracias a esta un día sin solicitudes produce un ítem vacío, y el nodo de IF detecta que no hay ruta, derivando a un correo en donde se pone explícitamente que no hubo solicitudes en ese día.

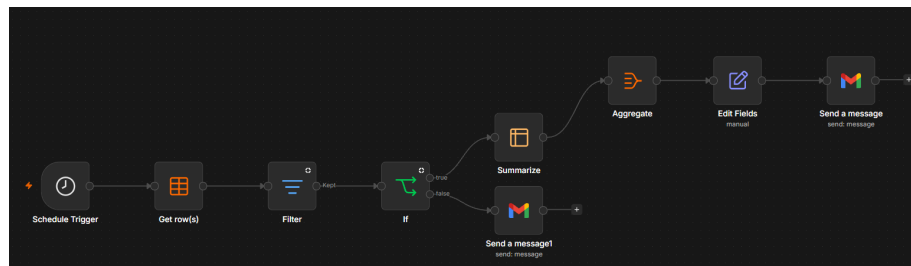


Figura 3: Arquitectura del flujo de Resume

3 Parte C

3.1 Entradas Invalidas

Cuadro 2: Entradas inválidas enviadas al flujo

Caso	Carga	Comportamiento esperado	Comportamiento observado
ERR-01	<code>id_solicitud</code> vacío	DATOS_INVALIDOS	Ruta DATOS_INVALIDOS, motivo Falta <code>id_solicitud</code>
ERR-02	<code>monto</code> con valor <code>-5000</code>	DATOS_INVALIDOS	Ruta DATOS_INVALIDOS, motivo con el monto recibido
ERR-03	<code>monto</code> no numérico	Falla de un nodo	Ejecución en error en el nodo Switch

- El ERR-03 rompe el nodo ya que la ejecución termina en error junto con el mensaje *cannot convert to number* en el nodo Switch, el dato llegó bien al nodo, el problema fue a la hora

de hacer la comparación dentro del Switch ya que se comparo un monto no numérico con uno numérico.

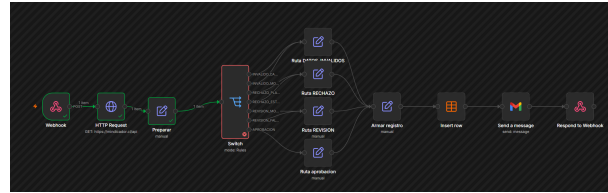


Figura 4: Error de conversión en el nodo Switch al recibir un monto no numérico.

3.2 Los tres valores de On Error

Valor	Comportamiento
Stop Workflow	Detiene la ejecución; queda registrada en rojo.
Continue (using error output)	Agrega una segunda salida por la que salen los ítems fallidos.
Continue (using regular output)	El ítem sigue por la salida normal con un campo error .

Los nodos centrales que poseen toda la trazabilidad del sistema (tales como: Preparar, Switch, las rutas, Armar registro, Registrar y Responder) quedaron en Stop Workflow, ya que si alguno de estos falla, la solicitud no se clasifica ni registra, ya que es mejor que aparezca el indicador visible rojo que una fila que nadie va a revisar hasta que sea demasiado tarde. Consultar Uf y Notificar usan Continue (using regular output), ya que no son de vital importancia, podemos prescindir del valor del UF y un correo rebotado no va a borrar o cambiar los registros. No se hizo uso de Continue (using error output) ya que simplemente no fue necesario para este sistema.

3.3 Error Workflow

Este tipo de workflow se ejecuta automáticamente cuando otro falla, recibiendo los datos de la ejecución fallida. Un flujo sin este workflow deja de funcionar sin que nadie se llegue a enterar, ya que el workflow corre sin que nadie lo este monitorizando y en caso de que algún nodo llegue a fallar, la ejecución queda en rojo en el historial y nadie lo termina revisando, ya que no se posee este workflow que puede convertir esa falla silenciosa en alguna notificación de error.

3.4 Casos Bordes creados

Caso	Condición	Ruta obtenida	Regla aplicada
LIM-01	$\text{monto} = U = 56.000$	APROBACION	Regla 6
LIM-02	$\text{dias_desde_compra} = D = 21$	APROBACION	Regla 6

Ambos casos quedan dentro de lo aceptado, cumpliendo la regla 6. El Switch se encarga de comparar con la función *is greater than* y no con *greater than or equal*, por lo que no dispara ninguna regla y cae directamente en el fallback. En caso contrario LIM-01 habria salido por REVISION y LIM-02 por RECHAZO, y la politica se habria aplicado erroneamente en este caso.

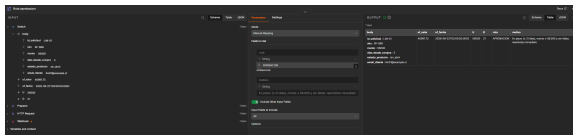
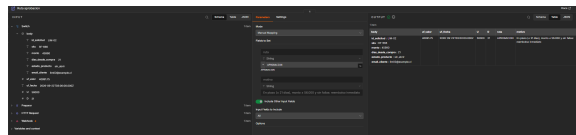
(a) LIM-01: monto igual a U (b) LIM-02: días iguales a D

Figura 5: Los dos casos de frontera quedan aprobados.

3.5 Error de tipo, a proposito

Al cambiar el tipo de la condición del monto de *Number* a *String* y enviar DEV-C01 (\$9.990), el workflow detiene el nodo con el mensaje “*Wrong type: '9990' is a number but was expecting a string [condition 0, item 0]*”: la expresión `.toNumber()` entrega un número cuando la condición espera texto. Tras revertir la condición a *Number*, Dev-C01 vuelve a clasificar como Aprobacion.

3.6 Los dos casos que el flujo no pudo resolver

"monto": "veinte mil". Los campos llegan en forma de texto y las reglas 2 y 5 comparan números, de modo que el Switch convierte con `.toNumber()` los números, pero cuando el texto no representa un número, la conversión falla y el nodo se cae. Para corregirlo el nodo necesitaría una validación antes del Switch (un IF) que compruebe con una expresión que el campo "monto" solo posee números y derivar a otra sección los que no cumplan este requisito.

"sku": "ZZ-999". La solicitud pasa la validación y se clasifica como APROBACION, porque ninguna de las seis reglas mira el SKU, ya que se cumplen todas las reglas y el SKU inventado obtiene un reembolso automáticamente, para corregirlo habria que cargar un catalogo y que busque el SKU antes del Switch, y agregar la regla de "SKU inexistente → DATOS_INVALIDOS".

3.7 Recomendacion sobre el MVP

El flujo resuelve de buena manera el problema para el que fue hecho. El atasco estaba en el triage, y ahora cada solicitud se procesa y clasifica en segundos, a diferencia de la demora de dias del anterior. A su vez tiene una mejor trazabilidad ya que deja un resumen diario de el movimiento. Aun con esto, lanzarlo oficialmente tal como esta es arriesgado, ya que como vimos anteriormente hay ciertos inputs que hace que genere distintos tipos de errores, como que no revise el contenido que escribe y que un SKU obtiene un reembolso automático.

Lo mejor seria adoptarlo como sistema definitivo después de hacer unas pruebas más exhaustivas y encontrar todas las falencias del actual y corregirlas, mientras eso no ocurra conviene seguir manteniendo este flujo andando pero realizar revisiones periódicamente a mano para encontrar ocasiones en donde el workflow tenga errores y atender esos casos por separado.

Bibliografía

- [1] N8N, *n8n Docs*, documentación oficial, <https://docs.n8n.io>.
- [2] N8N, *Switch node documentation*, <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.switch/>.
- [3] MINDICADOR.CL, *API de indicadores económicos de Chile*, <https://mindicador.cl/api>.